

Tarea 3 - 2024-2 Nicolas Hormann

Parte 1

1. Describe cómo sería una solución utilizando un enfoque "Greedy":

- Si dar el paso disminuye la profundidad del sueño por debajo de w , calculamos cuánto tiempo t necesitamos esperar para que la profundidad suba lo suficiente. Después de esperar t , avanzamos al siguiente paso. Si en algún momento t se vuelve imposible porque p sería superada devolvemos -1.

2. Investigue brevemente qué es la subestructura óptima de una solución. ¿Por qué este problema sigue la subestructura óptima? ¿Es esto útil para resolver este problema?

- Un problema tiene subestructura óptima si la solución óptima del problema puede obtenerse combinando soluciones óptimas de sus subproblemas.
- El problema sigue la subestructura óptima porque la profundidad del sueño del puffle depende solo del estado actual (profundidad restante) y no del orden de pasos anteriores, siempre que esperemos lo suficiente para no cruzar el umbral w . Esto significa que las decisiones óptimas en cada paso pueden reducirse a resolver el problema restante (los pasos subsiguientes) de manera óptima.
- Es útil porque esta característica nos permite usar un enfoque greedy.

Parte 2

1. Describe y justifica brevemente tu solución, puedes apoyarte de algún algoritmo visto en clases.

- Usamos Prim ya que con este método podemos encontrar de manera sencilla el árbol que conecta a todos los maestros y con costo mínimo. Use min-heap para optimizar la selección de la arista con menor costo en cada iteración.

2. Explica en qué casos no es posible conectar a todos los maestros y de qué manera puedes chequearlo en código.

- Si el grafo no es conexo es imposible conectar todos los vértices de este, entonces si no todos los nodos son marcados como visitados quiere decir que no es conexo esto se revisa (`src/card-jitsu/prim.c`, línea 157) que en caso de ser conexo retornamos el valor de la suma de las aristas recorridas.

3. Explica la complejidad temporal de tu solución en notación Big O.

- Construir el grafo $O(L)$
- Cada nodo se inserta/elimina del min-heap $O(N \log N)$ y cada arista se evalúa $O(L \log N)$ al usar min-heap.

Entonces la complejidad total es de $O(L \log N)$ ya que $L \geq N$ (en general)